Universidad de los Andes
Departamento de Ingeniería de Sistemas y Computación
Web Semántica (ISIS4514) ▪ 2026-20 ▪ Rubén Manrique

Guía de proyecto ▪ Etapa 2

De los datos consolidados a la ontología

Formalizar, extender con OWL, poblar y consultar en Fuseki

Sesión 10 ▪ jueves 8 de octubre de 2026 ▪ Acompaña al enunciado oficial de la Etapa 2

Para qué sirve esta guía

El enunciado oficial de la Etapa 2 dice *qué* hay que entregar. Esta guía dice *en qué orden conviene hacerlo* y dónde están las trampas, y sirve tanto para los setenta minutos de trabajo en clase como para los cuatro días siguientes.

El punto de partida son los dos productos de la Etapa 1, que son el modelo conceptual preliminar y el dataset consolidado de más de 75.000 canciones con sus fuentes e identificadores de correspondencia. El punto de llegada son `ontology.ttl` y `instances.ttl` cargados en Apache Jena Fuseki, con al menos cinco consultas SPARQL que responden otras tantas *competency questions*.

El enunciado permite modificar el modelo conceptual de la Etapa 1 cuando se encuentren problemas o ambigüedades, y exige documentar y justificar los cambios. Conviene aprovecharlo, porque el modelo preliminar se hizo antes de conocer OWL.

En caso de discrepancia entre esta guía y el enunciado oficial, manda el enunciado.

Al terminar esta lectura, usted debe poder

1. Revisar el modelo conceptual de la Etapa 1 y formalizarlo en RDF y RDFS, con clases, subclases, propiedades de objeto y de dato, dominios, rangos y jerarquías.
2. Extender el modelo con al menos ocho axiomas OWL de al menos tres tipos distintos, cada uno justificado por una característica real del dominio musical.
3. Redactar ocho *competency questions* y usarlas para revisar el modelo antes de poblarlo.
4. Generar las instancias por código, con URI consistentes y reproducibles, manteniendo separados el vocabulario y los datos.
5. Cargar el grafo en Fuseki y resolver al menos cinco de las preguntas con SPARQL, documentando los resultados.

1 Preparación

Cada grupo necesita, abierto antes de empezar,

- el modelo conceptual entregado en la Etapa 1, con sus clases, relaciones y jerarquías;

- el dataset consolidado de la Etapa 1, en uno o varios `.csv`, con la columna de procedencia y los identificadores de correspondencia;
- un entorno de Python con `rdflib`, que es el mismo del Laboratorio 2;
- Apache Jena Fuseki, el mismo del Taller 2.

> **Nota**
>
> No hace falta entregar un servidor Fuseki configurado. Lo que se entrega son los archivos y un `configuracion.py` con los parámetros de conexión centralizados, de modo que los profesores puedan cargar el grafo en su propia instalación y obtener los mismos resultados que estén documentados.

> **Cuidado** — El orden importa
>
> El enunciado pide que las *competency questions* se formulen *antes* de terminar el modelo, porque su función es revisarlo. Hacerlas al final, para cumplir el entregable, desperdicia el único mecanismo de validación que la etapa ofrece. Por eso en esta guía van en la sección 3, antes de poblar.

2 Trabajo en clase

Setenta minutos en cuatro bloques, con un corte anunciado entre cada uno. Los bloques cubren las decisiones que cuesta tomar en solitario; la escritura de código queda para después.

	Tiempo	Qué se hace	Producto del bloque
1	25 min	Redactar las ocho *competency questions*.	La tabla de la sección 3 llena.
2	15 min	Comprobar si el modelo de la Etapa 1 las soporta.	La lista de clases y propiedades que faltan o sobran.
3	20 min	Elegir los ocho axiomas OWL y justificarlos.	La tabla de la sección 6 llena.
4	10 min	Decidir el esquema de URI y el reparto en dos archivos.	La regla de construcción de URI, escrita.

Tabla 1: Los cuatro bloques de la sesión.

3 Competency questions

El enunciado pide **ocho**, planteadas pensando en un producto concreto construido sobre el grafo, por ejemplo un explorador musical, un recomendador, una herramienta para analizar colaboraciones entre artistas o un sistema de consulta sobre la industria musical. Decidir el producto primero ayuda, porque las preguntas salen solas.

> **Cuidado** — La prueba que tiene que pasar cada pregunta
>
> Antes de dar una pregunta por buena, compruebe las dos cosas. Que se puede escribir la consulta SPARQL que la responde sobre su modelo, y que se puede decir de antemano qué debería devolver. Si alguna de las dos falla, la pregunta habla del modelo o de los datos en abstracto en lugar de hablar del dominio musical, y hay que reformularla.

Ejemplo 3.1 — Tres preguntas comentadas

- *¿Qué información tenemos de los artistas?*
  No sirve, porque no restringe nada. Cualquier modelo la responde.
- *¿Qué artistas han colaborado, directa o indirectamente, con Miles Davis?*
  Sirve. Obliga a modelar la colaboración como relación entre artistas, y la palabra «indirectamente» obliga además a declararla transitiva o a recorrerla con un camino de propiedad.
- *¿Es buena la ontología?*
  No sirve, porque no se traduce a ninguna consulta sobre el grafo.

El enunciado pide además que las consultas no se limiten a recuperar atributos individuales, sino que involucren relaciones entre entidades, jerarquías, filtros o agregaciones. Conviene por tanto repartir las ocho preguntas entre los tres tipos de la tabla 2.

Tipo	Ejemplo en el dominio musical	Qué fuerza a modelar
Recuperación	¿Qué canciones de rock progresivo publicó el sello Harvest entre 1970 y 1975?	Las clases, el sello, el género y el año.
Navegación	¿Qué artistas han colaborado, directa o indirectamente, con Miles Davis?	La relación de colaboración y su transitividad.
Agregación	¿Cuál es el tempo medio de las grabaciones de cada subgénero del jazz?	La jerarquía de géneros y los descriptores acústicos.

Tabla 2: Los tres tipos, con ejemplos del dominio del proyecto.

Use esta plantilla, una fila por pregunta. Las ocho van en el documento de entrega, y al menos cinco de ellas se implementan en SPARQL.

#	Pregunta	Tipo	Respuesta esperada	SPARQL
1				
2				
3				
4				
5				
6				
7				
8				

Tabla 3: Plantilla del bloque 1.

4 Revisar el modelo con las preguntas

Tome cada pregunta y señale sobre el modelo de la Etapa 1 las clases y propiedades que la consulta necesitaría. De ahí salen tres listas.

1. Lo que el modelo ya tiene y la pregunta usa.
2. Lo que falta. Son las clases y propiedades que hay que añadir, y es el resultado más valioso de este ejercicio.
3. Lo que el modelo tiene y ninguna pregunta usa. Conviene preguntarse si sostiene algún requisito no escrito o si se puede retirar.

> **Nota**
>
> El enunciado dice que el modelo conceptual puede modificarse en esta etapa y que los cambios deben documentarse y justificarse. Las tres listas de arriba son exactamente esa documentación, de modo que conviene conservarlas tal como salgan de este bloque.

5 Formalizar en RDF y RDFS

La ontología se genera por código, de forma reproducible, y se exporta en Turtle. Los elementos que el enunciado pide definir son clases y subclases, propiedades de objeto, propiedades de dato, dominios y rangos, jerarquías e instancias.

Ejemplo 5.1 — Un fragmento del vocabulario

ontology.ttl, extracto

```
:Artista   a owl:Class ; rdfs:label "Artista"@es .
:Grupo     a owl:Class ; rdfs:subClassOf :Artista .
:Grabacion a owl:Class ; rdfs:label "Grabación"@es .
:Genero    a owl:Class ; rdfs:label "Género musical"@es .

:interpreta a owl:ObjectProperty ;
   rdfs:domain :Artista ; rdfs:range :Grabacion .

:tempo a owl:DatatypeProperty ;
   rdfs:domain :Grabacion ; rdfs:range xsd:decimal .
```

> **Cuidado** — Dominio y rango no validan
>
> Declarar `:interpreta rdfs:domain :Artista` no rechaza los datos en los que el sujeto no sea un artista. Lo que hace es *inferir* que ese sujeto es un artista, según la condición de modelo de la Lectura 6. Con datos integrados de varias fuentes, como los de la Etapa 1, esto propaga tipos a entidades mal reconciliadas sin que nada avise. La validación con rechazo se verá con SHACL en la sesión 11.

6 Extender con OWL

El enunciado pide **al menos ocho axiomas o restricciones**, usando **al menos tres tipos distintos** de construcción, y advierte que deben responder a características reales del dominio y no incorporarse solo para llegar al número. Cada uno se explica brevemente y se justifica.

Construcción	Ejemplo en el dominio musical	Qué afirma
Clases disjuntas	`:Grabacion owl:disjointWith :Artista`	Ninguna cosa es a la vez grabación y artista.
Propiedades inversas	`:interpreta owl:inverseOf :interpretadaPor`	Las dos relaciones son la misma vista al revés.
Propiedad simétrica	`:colaboraCon a owl:SymmetricProperty`	Si A colabora con B, B colabora con A.
Propiedad transitiva	`:subgeneroDe a owl:TransitiveProperty`	El bebop es subgénero del jazz por serlo de uno de sus subgéneros.
Propiedad funcional	`:tieneDuracion a owl:FunctionalProperty`	Una grabación tiene una sola duración.
Propiedad inversa funcional	`:tieneMBID a owl:InverseFunctionalProperty`	Dos entidades con el mismo MBID son la misma.
Cardinalidad	`:Album` con al menos una `:tienePista`	Un álbum sin pistas no es un álbum.
Clase equivalente	`:ArtistaPremiado` ≡ `:Artista` ⊓ ∃`recibio.Premio`	Todo artista del que conste un premio queda clasificado como premiado.
Restricción sobre valores	`:AlbumDeEstudio` con ∀`tienePista.GrabacionDeEstudio`	Todas las pistas de un álbum de estudio son grabaciones de estudio.

Tabla 4: Las construcciones que el enunciado enumera, con un ejemplo del dominio cada una. Hay que usar al menos tres tipos distintos.

Cuidado — El axioma que más rinde y el que más daño hace

`owl:InverseFunctionalProperty` sobre un identificador externo, como el MBID de MusicBrainz, hace que el razonador concluya `owl:sameAs` entre las entidades que lo comparten. Es la forma más limpia de aprovechar la reconciliación hecha en la Etapa 1, y es exactamente lo que se quiere cuando los identificadores están limpios.

Cuando no lo están, el mismo axioma fusiona entidades distintas y propaga todas las propiedades de una a la otra, y el síntoma aparece lejos de la causa. Si el proceso de la Etapa 1 dejó identificadores dudosos, conviene declararlo sobre el identificador en el que más confianza se tenga y comprobar el efecto sobre una muestra antes de cargar el grafo completo.

Cuidado — Contar bajo ausencia de nombres únicos

Declarar que una grabación pertenece a exactamente un álbum no rechaza los datos en los que aparezca en dos. Lo que hace el razonador es concluir que los dos IRI de álbum denotan el mismo álbum, según la Lectura 6. Como una misma grabación aparece de hecho en recopilatorios y reediciones, esta restricción es casi siempre falsa en el dominio musical y conviene no ponerla.

Nota — Primitiva o definida

Una clase escrita con `rdfs:subClassOf` solo deduce hacia abajo. Una escrita con `owl:equivalentClass` además clasifica individuos hacia arriba, que es lo que hace visible el trabajo del razonador. Conviene que al menos una de las ocho sea definida, y la fila de `:ArtistaPremiado` de la tabla 4 es el patrón.

Use esta plantilla para los ocho, porque las tres columnas son justamente lo que el documento de entrega tiene que contener.

#	Tipo de construcción	Axioma	Por qué es verdadero en el dominio
1			
2			
3			
4			
5			
6			
7			
8			

Tabla 5: Plantilla del bloque 3. Al menos tres tipos distintos en la segunda columna.

7 Generar las instancias

El enunciado pide mantener separados el vocabulario y los datos, en `ontology.ttl` y `instances.ttl`, y generar las instancias automáticamente a partir del dataset de la Etapa 1.

7.1 El esquema de URI

Los identificadores tienen que ser consistentes y reproducibles. Dos ejecuciones del código sobre los mismos datos tienen que producir los mismos IRI.

Ejemplo 7.1 — Una regla de construcción de URI

Patrón de IRI, con el identificador entre ángulos

```
http://ejemplo.org/musica/recurso/artista/<mbid>
http://ejemplo.org/musica/recurso/grabacion/<msd-track-id>
http://ejemplo.org/musica/recurso/genero/<nombre-normalizado>
```

Cada IRI se deriva de un identificador estable del dataset. Cuando una entidad no tenga identificador externo, conviene derivarlo de una huella de los campos que la identifican, por ejemplo el resumen de la concatenación normalizada de título y artista.

Cuidado

Generar los IRI con `uuid4()`, con el número de fila o con un contador rompe la reproducibilidad que el enunciado exige, y lo hace de forma silenciosa, porque el archivo parece correcto. El síntoma aparece al volver a ejecutar el código y encontrar un grafo entero de entidades nuevas.

7.2 Qué conviene comprobar antes de cargar

1. Que los dos archivos parsean con `rdflib`.
2. Que el número de instancias de `:Cancion` se corresponde con el número de filas del dataset consolidado, que el enunciado espera por encima de 75.000.

3. Que ninguna entidad tiene dos IRI distintos por una diferencia de mayúsculas o de espacios en el campo del que se deriva.
4. Que un razonador declara el grafo consistente. Una inconsistencia aquí casi siempre viene de una clase disyunta declarada de más o de un identificador compartido por error.

8 Cargar y consultar en Fuseki

Se cargan los dos archivos en Apache Jena Fuseki y se implementan al menos cinco de las ocho preguntas en SPARQL. Las consultas deben usar la estructura del modelo, con relaciones entre entidades, jerarquías, filtros o agregaciones, y no limitarse a recuperar atributos sueltos.

Ejemplo 8.1 — Una consulta que usa la estructura

```
SELECT ?subgenero (AVG(?t) AS ?tempoMedio) WHERE {
  ?subgenero :subgeneroDe+ :jazz .
  ?grabacion :tieneGenero ?subgenero ;
             :tempo       ?t .
} GROUP BY ?subgenero
```

Recorre la jerarquía de géneros con un camino de propiedad, cruza dos entidades y agrega. Una consulta que solo pidiera el tempo de una grabación no demostraría nada del modelo.

> **Nota** — El archivo de configuración
>
> `configuracion.py` centraliza el nombre del dataset y las URL de los *endpoints* de consulta y de actualización. Los profesores van a cargar `ontology.ttl` e `instances.ttl` en su propia instalación de Fuseki con esa configuración, y a ejecutar las consultas esperando obtener los mismos resultados que estén documentados. Conviene por tanto documentar el resultado real, y no el que se esperaba obtener.

9 Lo que esta etapa no pide

El enunciado excluye de la Etapa 2 la vinculación con ontologías externas y con fuentes de Linked Data, que se aborda más adelante en el curso. En concreto, en esta entrega no hay que buscar vocabularios en LOV, ni importar nada con `owl:imports`, ni alinear la ontología propia con ninguna ajena, ni publicarla con IRI resolubles.

> **Nota**
>
> Conviene de todos modos elegir los nombres de las clases y propiedades pensando en que esa vinculación va a ocurrir, porque un vocabulario cuyos términos siguen las convenciones habituales se alinea después con mucho menos trabajo.

Tampoco se pide análisis de audio. Los descriptores acústicos, cuando se usen, provienen de fuentes ya existentes, según establece la Etapa 1.

10 Antes de entregar

Esta es la lista de entregables del enunciado oficial, en forma de lista de verificación.

1. Documento de **máximo 6 páginas** con las decisiones de modelamiento, los cambios hechos sobre el modelo de la Etapa 1 con su justificación, las construcciones RDFS y OWL usadas, las ocho *competency questions* y los problemas encontrados.

2. `ontology.ttl` con la definición formal de la ontología.
3. `instances.ttl` con las instancias generadas a partir de los datos consolidados, y el código que lo genera.
4. Código fuente de la transformación, en un repositorio de GitHub **público**.
5. El listado de las ocho *competency questions*.
6. Las consultas SPARQL de al menos cinco de ellas, con los resultados obtenidos al ejecutarlas.
7. Evidencia de la ontología y las instancias cargadas y funcionando en Apache Jena Fuseki, con `configuracion.py`.
8. Si se hizo el bonus, el código de extracción desde texto y la descripción de las entidades, propiedades o relaciones nuevas que incorporó al grafo.

Cuidado — Los cuatro descuidos más probables

1. Formular las *competency questions* al final, con lo que dejan de servir para revisar el modelo y se nota en que todas se responden sin esfuerzo.
2. Incorporar axiomas OWL que no son verdaderos en el dominio para llegar a ocho. El enunciado dice expresamente que eso no se evalúa de forma positiva.
3. Generar IRI no reproducibles, con lo que la segunda ejecución produce un grafo distinto.
4. Mezclar vocabulario e instancias en un solo archivo.